

e Yönelimli Program

Hafta 1: Dersin Amacı, OOP Vizyonu ve C++ Temelleri

s İşleyişi: Devam ve Zamanında Gelme

ve laboratuvara zamanında katılım, oturum düzeninin ve uygulamalı çalışmaların ön koşuludur.

ik Dersler (2 Saat):
demik Disiplin ve Zaman Yönetimi:

ayramsal altyapı, bellek anatomisi, nesne yönelimli tasarım prensipleri ve mimari analizler.
ders başlangıcından sonra amfiye/laboratuvara giriş ders akışını ve odaklanmayı belirler.

laboratuvar çalışmalarında zamanında bulunmayan öğrenciler pratik çalışma puanlamasından n
Uygulamalı Laboratuvar (2 Saat):

alır
odlama pratikleri, canlı hata ayıklama (debugging), haftalık alıştırmalar ve vaka analizleri.

Akademik Dürüstlük ve Kod Paylaşım İlkeleri

Akademik Özgünlük İlkesi:

Aftalık laboratuvar çalışmaları ve sınav çözümleri öğrencinin kendi bireysel zihinsel emeğinin ürünü olmalıdır.

Yapay Zeka ve Çevrim İçi Kaynak Kullanımı:

Yapay Zeka Araçları ve Çevrim İçi Kaynaklar:

Yapay zeka araçları kodun mantığını anlamak için bir öğrenme asistanı olarak kullanılabilir; ancak sonuçları ve algoritmaları arkadaşlarınızla tartışınız; ancak kod editörünüzdeki her satırı bizzat kendiniz yazarak kopyalamak öğrenme sürecini engeller.

Özgünlük Denetimi:

ta 1: Hedefler ve OOP Vizyonu

anın temel teorik hedefleri:

ne Yönelimli Programlama (OOP) Vizyonu:

ınıf (Class) ile Nesne (Object) arasındaki mimari ayrımı netleştirmek.
Procedure programlama © ile Nesne Yönelimli Programlama (C++) arasındaki kavramsal
ayramak.

Dili ve Mimarisi:

erçek hayat mekatronik sistemlerinin durum (state) ve davranış (behavior) üzerinden modeller
++ dilinin tasarım felsefesini (*Zero-overhead Principle*) ve donanım kontrolündeki gücünü anlamak.

öğrenmek.

nişlemci, derleyici ve bağlayıcıdan oluşan derleme mimarisini kavramak.

Hafta 1: Uygulamalı C++ Hedefleri

Haftanın laboratuvar ve kodlama kazanımları:

Derleme Ortamı ve Derleme:

C++ temel veri tiplerini, bellek boyutlarını (`sizeof`) ve değer aralıklarını kavramak.
Komut satırından `g++` ile C++ programını derleyip çalıştırmak.

Modern C++ süslü parantez `{}` ile liste inisiyalizasyonu (*Uniform Initialization*) standardını uygulama
Derleyici bayraklarının (`-std=c++14`) kullanımı amacını öğrenmek.

Giriş ve Çıkış Akışları
İşkenler ve Bellek Mimarisi:

`std::cout` , `std::cin` ve `std::cerr` akışlarını yönetmek.

Haftalık Ders İzlenesi (1/3: C++ Temelleri Bölüm 1)

Konu Başlığı	Odak Noktası ve Temel Kazanımlar
Konu Başlığı	Odak Noktası ve Temel Kazanımlar
OOP Vizyonu ve C++	Prosedürel vs OOP, derleme mimarisi, temel tipler, <code><iomanip></code>
Kontrol Yapıları ve Temelleri	Karar yapıları (<code>if-else</code> , <code>switch-case</code>), döngüler, Sonlu Durum Makinesi (FSM)
Döngüler	

Haftalık Ders İzlenesi (2/3: Nesne Yönelim Programlama)

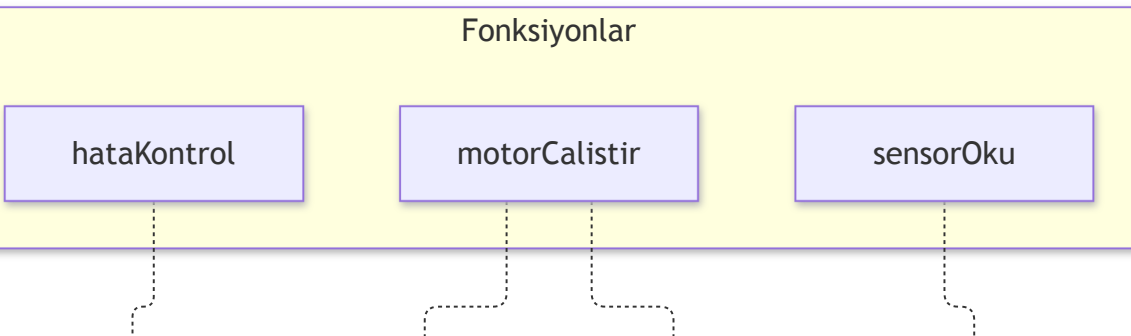
Konu Başlığı	Odak Noktası ve Temel Kazanımlar
Konu Başlığı	Odak Noktası ve Temel Kazanımlar
Yapılar, Birlikler ve Numaralandırma	struct , union , bit alanları, bayt hizalama/paketleme, enum
OOP'ye Giriş: Sınıflar ve Nesneler	Sınıf/nesne, erişim belirleyicileri, yapıcılar, ilklendirme listesi, friend

Haftalık Ders İzlenesi (3/3: İleri OOP, V ıları ve Modern C++)

Konu Başlığı	Odak Noktası ve Temel Kazanımlar
Konu Başlığı	Odak Noktası ve Temel Kazanımlar
Soyut Sınıflar, Arayüzler ve	Saf sanal metotlar, saf arayüz (<code>interface</code>), <code>override</code> ,
Dönüşüm	Aritmetik, akış (<code><<</code> , <code>>></code>), indeks (<code>[]</code>), atama (<code>=</code>)
Operatörlerin Aşırı Yüklenmesi	<code>dynamic_cast</code> , operatörleri

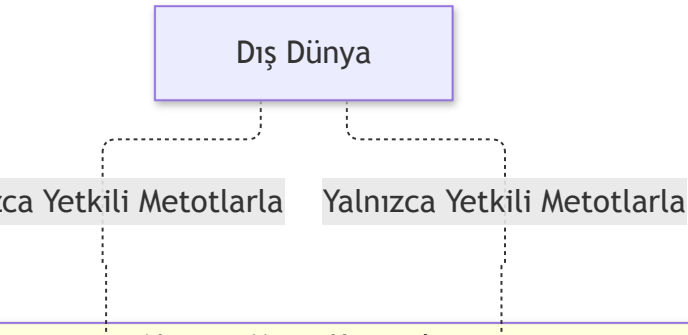
sedürel Programlama Mimarisi

onlar ve Veriler birbirinden ayırır:



Nesne Yönelimli Programlama (C++) Mimarisi

ve Fonksiyonlar nesne içinde paketlenir:



Sone Yönelimli Programlamanın Dört Temel Taşı

Modern OOP mimarisi 4 sütun üzerine inşa edilmiştir:

Kapsülleme (Encapsulation):

Kapsülleme, sınıflın içi ve dışı arasındaki farkı belirler. Sınıflın içi, yani sınıfın kendisi, verileri ve bu verileri işleyen metotları bir arada paketlenmesi, iç durumun dış müdahalelerden gizlenmesi (Data Hiding).

Varislik / Miras (Inheritance):

Varislik, bir sınıftan yeni sınıflar türeterek kodun yeniden kullanılması (*Code Reuse*) ve h

Kinematik Modelleme Örneği: Robot Kolu

Örnek: Robot Kolu

Özellikler / State):

Özellikler / State): Robot kolunun anlık fiziksel büyüklükleri üye değişkenlerde, kinematik hareket kabiliyeti ise üye metotlarda

Yöntemler / Methods):

hareketEt() , tutucuKapat() , acilDurdur()

Elektronik Modelleme Örneği: Sıcaklık Sensörü

Örnek: Sıcaklık Sensörü (LM35 / PT100)

Özellikler / State:

Notu: Ham ADC okuması sınıf içinde kapsülленir; dış modüllere doğrudan filtrelenmiş ve kalibre edilmiş sıcaklık verisi sunulur.

Yetenekler / Methods:

`sicaklikOku()`, `kalibreEt()`, `esikKontrol()`

Elektronik Modelleme Örneği: Otonom Araç

Örnek: Otonom Araç

Durum (State): Batarya seviyesi, direksiyon açısı, lidar engel mesafesi, GPS konumu
Dünya nesnenin batarya seviyesini rastgele değiştiremez; yalnızca `sarjEt()` metoduyla yönetilebilir.

Davranış (Behavior): `rotaHesapla()` , `seriTakipEt()` , `frenYap()` , `telemetryGonder()`

Tasarım Kuralı: Bir nesne, kendi verisini (state) ve bu veriyi işleyen fonksiyonlarını (behavior) tek bir çatı altında birleştirir.

Sınıf (Class) Kavramı: Taslak ve Şablon

Nesnenin taslağıdır (blueprint).

Yalnızca çizim ve tip kalıbıdır: nesne oluşturulana dek bellekte yer kaplamaz (statik üyeler hariç).

```
    int rpm;  
    void calistir();  
;
```

/ Sınıf Tanımı (Taslak / Şablon)

```
class Motor {  
public:
```

Nesne (Object) Kavramı: Bellekteki Somut Varlık

tan türetilen somut varlıktır (instance).

şöyle bir nesne inşa edilmiş gerçek yapıdır: bellekte (RAM) yer tahsis edilir.

```
Motor m2; // Bağımsız ikinci motor nesnesi

m1.rpm = 1500;
m2.rpm = 3000;
return 0;
// Nesne üretimi (Somutlama - Instantiation)
Motor m1; // RAM'de m1 için yer tahsis edildi
```

Elektronik Örneği: Motor Sınıfı ve Nesne Ayrımı

Robotik sistemde birden fazla motor bağımsız kontrol edilecekse:

```
class Motor {  
public:  
    int pwmPini;  
    int maxHiz;  
    int anlikHiz;  
};  
Motor sagIleker;
```

Motor Davranışları: Üye Fonksiyonlar (Metotlar)

Motor nesnesi ortak üye fonksiyonları kendi üye değişkenleriyle yürütür:

```
class Motor {  
public:  
    void durdur() {  
        anlikHiz = 0;  
    }  
    void hizAyarla(int yeniHiz) {  
        anlikHiz = yeniHiz;  
    }  
};
```

Örneğin: `Motor m1; m1.hizAyarla(100);` → `m1` nesnesinin `anlikHiz` üyesi güncellenir.

- Dili: Tarihçe ve Standartlar

ne Stroustrup (1979 - Bell Labs):

dilinin donanıma doğrudan erişim hızını, Simula dilinin nesne yönelimli mimarisiyle birleştirdi (

lasses")
++11 (Modern C++ Devrimi): `auto`, akıllı işaretçiler (`smart pointers`), taşıma semantiği (

1983 yılında resmi olarak C++ adını aldı (`++` artırma operatörü).

Eyrimi ve Standartları:

++14 / C++17: Genişletilmiş `constexpr`, `std::optional`, `std::variant`, dosya sis
filesystem>); ilk ISO standardı, Şablonlar (Templates), Standart Şablon Kütüphanesi (STL).

++20 / C++23: Modüller (`modules`) kavramlar (`concepts`) coroutines, ranges kütüphanesi

Neden C++? Mekatronik ve Gömülü Sistemlerde Erişimi

Mekatronik mühendisliğinde C++ neden vazgeçilmezdir?

Doğrudan Donanım Erişimi:

Ek Yük Prensibi (Zero-overhead Principle):

Yaratıcılar ve bellek adresleriyle mikrodenetleyici register'larına doğrudan yazabilme.

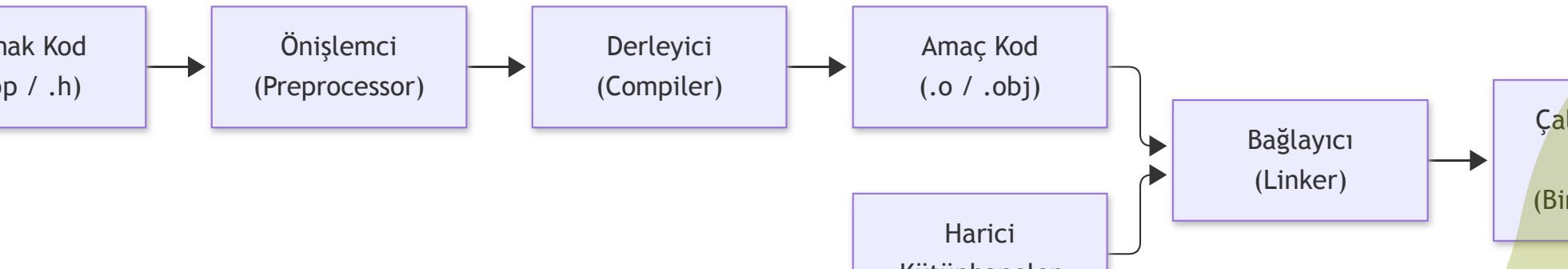
Kullanmadığınız şeyin bedelini ödemezsiniz. Kullandığınız şeyi ise elinizle yazsanız daha

denimci Bellek Yönetimi (Deterministic RAM):

Çöp toplayıcı (Garbage Collector) gecikmesi yoktur; nesne kapsamdan çıktığı mikro-saniyede

- Derleme Mimarisi (Aşamalar)

ğrudan çalıştırılan bir dil değildir; kaynak kod makine diline derlenir:



- Derleme Aşamaları: Karşılaşılan Hata Türleri

Hata hangi aşamada tespit edilir?

Derlemci Hataları:

Sözdizimi (syntax) hataları (eksik noktalı virgül, kapatılmamış parantez).
Bulunamayan başlık dosyaları (fatal error: Iostream: No such file).

Tür uyumsuzlukları ve geçersiz tür dönüşümleri.
Tanımlanmamış makro tanımlamaları.

Tanımlanmamış değişken veya fonksiyon kullanımı.

Derleme Zamanı Hataları (Compiler Errors):

Bağlayıcı Hataları (Linker Errors):

Her Satırdan Derleme (Tek Dosya)

Yayın dosyasını doğrudan terminalden derlemek:

```
Temel derleme  
Programı çalıştırma  
++ main.cpp -o program  
/program
```

```
Tüm uyarıları açarak derleme (Önerilen Akademik Standart)  
++ -Wall -Wextra -std=c++14 main.cpp -o program
```

lu Kaynak Dosyası Derleme ve Bağlayıcı (Linker)

n haftalarda sınıflarımızı başlık (`.h`) ve kaynak (`.cpp`) dosyalarına ayıracağız:

İki kaynak dosyasını birlikte derleyip bağlama:

```
g++ Motor.cpp main.cpp -o robotProgrami  
++ Motor.o main.o -o robotProgrami
```

Ayrı ayrı amaç koda dönüştürüp bağlama:

```
++ -c Motor.cpp -o Motor.o
```

Geliştirme Ortamı Kurulumu (IDE ve Editörler)

Önerilen ve desteklenen geliştirme araçları:

Editor Seçenekleri:

IntelliJ IDEA (Google): Yapay zeka destekli geliştirilmiş bir IDE'dir. LLM'ler ile entegre edilmiş VS Code. Mali, modüler, C/C++ uzantısı (Microsoft) ile güçlü entegrasyon. (Önerilen)

Code::Blocks: Kod yazmayı kolaylaştırır. (Önerilen)
Lion: JetBrains tarafından geliştirilen tam donanımlı C++ IDE'si (öğrenci lisansı ile ücretsiz).

Visual Studio (Windows): Windows ortamında MSVC derleyicisi ile eksiksiz araç seti.
Eklenti Eklentiler (VS Code):

C/C++ (Microsoft)

Derleyici Kurulumu ve Doğrulama

alden `g++ --version` komutu ile derleyici varlığını doğrulayınız:

x (Ubuntu/Debian):

```
xcode-select --install  
# veya Homebrew ile:  
brew install gcc
```

OS:
dows:

VS2019 / MinGW-w64: GCC/C++'sini kurun

Program: Merhaba Dünya!

```
include <iostream>
```

```
nt main() {  
    // Ekrana standart çıktı yazdır  
    std::cout << "Merhaba. Marmara Mekatronik!" << std::endl;  
++ -std=c++14 -Wall merhaba.cpp -o merhaba  
/merhaba
```

Çıktısı:

Program Yapısının Satır Satır Analizi

```
#include <iostream> :
```

giriş/çıkış akış kütüphanesini (*Input/Output Stream*) programa dahil eder.

standart karakter çıktısı nesnesi (*Character Output*). Ekrana veri akışı sağlar.

```
int main( ) :
```

programın ana giriş fonksiyonudur; işletim sistemi yürütmeye bu noktadan başlar.
(Akış Ekleme Operatörü).

çıkıştaki veriyi solundaki çıktı akışına yönlendirir.

```
cout << :
```

```
endl :
```

Yayın Hata Mesajlarının Analizi

Yayın hatası içeren kod örneği:

```
include <iostream>
int main() {
    std::cout << "Test" // Noktalı virgül EKSİK!
main.cpp: In function 'int main()':
main.cpp:4:5: error: expected ';' before 'return'
  4 |     return 0;
    |     ^~~~~~
```

1. Mesajı Okuma Stratejisi

`main.cpp` : Hatanın tespit edildiği kaynak dosya adı.

`main.cpp` : Satır 4. Sütun 5 (Derleyici hatayı noktalı virgülün eksikliğini fark ettiği bir sonraki satırın k
ected ';' before 'return' : Hatanın açık nedeni: `return` ifadesinden önce `;` k
arı.
eniyordu.

`main.cpp` : Derleme sürecini durduran kritik hata; çalıştırılabilir dosya üretilmez.

Önemli Notu: Derleyici hata listesini incelerken her zaman en baştaki ilk hatadan başlayınız. İlk satırdaki tek bir noktalı virgülü derleyicinin durumunu bozarak onlarca sahte ardışık hataya yol açabilir.

um Satırları ve Kod Dokümantasyonu

```
/ Tek satırlık açıklama
```

```
*  
// @param pwm 0-255 arası hız değeri  
*  
* açıklama bloğu  
*/
```

```
// Doxygen formatında dokümantasyon  
// @brief Motorun PWM değerini ayarlar
```

- Temel Veri Tipleri (Temel Türler)

Her değişken tanımlanırken veri tipinin açıkça bildirilmesi zorunludur (*Statically Typed*):

	Tip Adı	Boyut (Tipik)	Değer Aralığı
Yeni	int	4 Bayt (32 bit)	$-2.1 \times 10^9 \dots + 2.1 \times 10^9$
Uzun Tamsayı	long	8 Bayt (64 bit)	$\approx \pm 9 \times 10^{18}$
Kısa Tamsayı	short	2 Bayt (16 bit)	$-32,768 \dots + 32,767$
Yüzerlik	float	4 Bayt (32 bit)	≈ 7 basamak ondalık hassasiyet

Değişken Tanımlama Kuralları

İnde değişken tanımlarken uyulması gereken temel kurallar:

Herli İsimlendirme Kuralları:

İsimlendirme Temelleri
Her değişken, fonksiyon veya yapı tanımlama satırı (`int` , `void` , `struct`) ile başlamalıdır.

Değişken ve fonksiyonlar için: camelCase (`motorHizi` , `sensorDegeri`)r1 , `motor_2`).
Sabitler için: PascalCase (`MOTOR_KONTROL_CU` , `ROBOT_KOLU`)

Boşluk ve özel karakterler için: boşluk ve özel karakterler kullanılmamalıdır.

Sabitler için: `UPPER_SNAKE_CASE` (`MAX_HIZ` , `PI_SAYISI`)e `HIZ` üç farklı değişkendir.

Değişken Tanımlama: Geçersiz İsimler

Şu tanımlamalar derleme hatası üretir:

1. `int 1sensor = 10;` → Değişken ismi rakamla başlayamaz!

2. `float class = 5.0;` → C++ anahtar kelimeleri (`class` , `int` , `return`) değişken isimleri olamaz!

3. `double motor-hizi;` → Tire `-` eksi operatörü olarak yorumlanır!

→
4. Değişken adlarında Türkçe karakter (``ç`` , ``ğ`` , ``ı`` , ``ö`` , ``ş`` , ``ü``) kullanmayınız; derleyici uyumluluğunu bozar.

sizeof Operatörü ve Bellek Boyutları (C++ Kodu)

İ tipinin bellekte kapladığı bayt miktarını derleme zamanında döndürür:

```
include <iostream>
std::cout << "float:          " << sizeof(float)          << " bayt\n";
nt main() {
std::cout << "double:         " << sizeof(double)         << " bayt\n";
std::cout << "long:           " << sizeof(long)           << " bayt\n";
return 0;
std::cout << "char:            " << sizeof(char)            << " bayt\n";
std::cout << "short:           " << sizeof(short)           << " bayt\n";
std::cout << "int:             " << sizeof(int)             << " bayt\n";
```

Endriye Gre Veri Tipi Boyutları

64-bit mimarisindeki tipik konsol ıktısı:

bool:	1 bayt
byte:	1 bayt
short:	2 bayt
int:	4 bayt
float:	4 bayt

Değişken İlklendirme Biçimleri (Initialization)

Bir değişkene ilk değer atamanın üç farklı yöntemi bulunur:

/ 1. Kopyalama İlklendirmesi (C-tarzı)

```
int a{30};  
int d{}; // Varsayılan değer atanır: 0
```

/ 2. Doğrudan İlklendirme (Direct Initialization)

```
int b(20);
```

/ 3. Liste İlklendirme (Modern C++11 - Uniform Initialization)

Modern C++ Liste İlkendirme (Uniform Initialization)

Süslü parantez `{}` ilkendirmesi tercih edilmelidir?

Daraltıcı Dönüşümü (Narrowing Conversion) Engeller:

```
int x = 5.7;    // Derlenir; değer 5'e yuvarlanır, kesirli kısım KAYBOLUR!  
int y{5.7};    // DERLEME HATASI: Daraltıcı dönüşüm derleme zamanında engellenir!
```

İnicialanmamış Değişkenler ve Çöp Değer Riski

İnicial değişkene ilk değer verilmediğinde bellekte ne oluşur?

```
include <iostream>
return 0;

nt main() {
    int sensorDegeri; // İlk değer atanmadı!
    std::cout << "Deger: " << sensorDegeri << "\n";
}
```

Deger: 32767 # veya 0, veya -1045239... (Öngörülemez)

Belirli Davranış (Undefined Behavior)

Unumacı Kodlama

Değerleri her zaman başlangıç değeri vererek tanımlayınız:

```
int sensorDegeri{0}; // Güvenli: 0 ile başlar
double sicaklik{25.0}; // Güvenli: 25.0 ile başlar
bool aktifMi{false}; // Güvenli: false ile başlar
```

Read-Only Değişkenler: `const` Anahtar Kelimesi

program boyunca değiştirilemez olan değişkenler `const` ile tanımlanır:

```
const double PI = 3.1415926535;  
const int esik; // DERLEME HATASI: uninitialized const 'esik'  
const int FULL_OF_FIN = 4,  
  
PI = 3.14; // DERLEME HATASI: assignment of read-only variable 'PI'
```

`const` değişkenler bildirildikleri anda ilklendirilmek zorundadır:

Enst Kullanımının Tasarımsal ve Donanım daları

ların Derleme Zamanında Yakalanması:

eyici Optimizasyonu:

anlışlıkla kritik bir donanım limitinin değiştirilmesini derleyici önler.

erleyici sabit değeri doğrudan makine talimatlarına gömebilir ve bellek erişim maliyetini düşürür.

Okunabilirliği ve Niyet Bildirimi:

odu inceleyen mühendis ilgili büyüklüğün sistem sabiti olduğunu net biçimde anlar.

Otomatik Tip Çıkarımı: `auto` (C++11)

C++11 ile derleyici, atanan başlangıç değerinden türü otomatik çıkarabilir:

```
auto x = 10;           // int olarak belirlenir
auto y = 3.14;         // double olarak belirlenir
// auto = int olarak belirlenir
auto sayi = 5;         // int türü belirlendi
sayi = "Metin";        // DERLEME HATASI!
```

`auto` dinamik bir tip (Python turu) değildir.

to Kullanım Kuralları ve Tercih Kriterleri

başlangıç değeri zorunludur:

```
auto gecersiz; // DERLEME HATASI: Tip çıkarımı için başlangıç değeri şarttır!  
  
// Uzun ve karmaşık tür bildirimi yerine:  
std::vector<int>::const_iterator it = vec.cbegin();  
// auto ile sade ve açık sözdizimi:  
auto it = vec.cbegin();
```

Basit değişkenlerde (`int sayi{10};`) türün açıkça yazılması kodun doğrudan kavranması aç

Matematik Operatörler

İndeki temel matematiksel işlem operatörleri:

Operatör	Anlamı	Örnek	Sonuç (a=10, b=3)
Operatör	Anlamı	Örnek	Sonuç (a=10, b=3)
+	Toplama	$a + b$	13
*	Çarpma	$a * b$	30
-	Çıkarma	$a - b$	7
/	Bölme	a / b	3 (tamsayı bölmesi!)

Operatörlerde İşlem Önceliği

Operatörleri belirli bir öncelik sırasına göre işletilir:

Parantezler: `()` → En yüksek öncelik

Ekme ve Çıkarma: `+`, `-`

(Unary) Operatörler: `++`, `--`, `+`, `-`, `!`

Karşılaştırma Operatörleri: `<`, `<=`, `>`, `>=`

Çarpma, Bölme, Modül: `*`, `/`, `%`

Eşliklik Operatörleri: `==`, `!=`

Tam Sayı Bölmesi Tuzağı ve Veri Kaybı Riski

İki tamsayı bölündüğünde sonuç daima bir tamsayıdır (kesirli kısım atılır):

```
int a = 5;  
int b = 2;  
double sonuc = a / b; // Sonuç: 2.0 (2.5 DEĞİL!)
```

`b` aritmetik işlemi yürütüldüğünde ara sonuç `2` tamsayısı olarak üretilir.

Yeni Tü DönüŖümü: static_cast

ondalık sonu elde etmek için iŖlenenlerden en az biri ondalıklı tipe dönüŖtürülmelidir:

```
int a = 5;  
int b = 2;
```

```
// Modern C++ standardı: static_cast  
double sonuc = static_cast<double>(a) / b; // Sonu: 2.5
```

```
static_cast<HedefTip>(ifade) :
```

Kişisel (Karşılaştırma) Operatörleri

iki değeri karşılaştırır ve `bool` türünde sonuç üretir (`true` / `false`):

Operatör	Açıklama	Örnek	Sonuç (<code>x = 5</code>)
Operatör	Açıklama	Örnek	Sonuç (<code>x = 5</code>)
<code>==</code>	Eşit mi?	<code>x == 5</code>	<code>true</code>
<code><</code>	Küçük mü?	<code>x < 10</code>	<code>true</code>
<code>!=</code>	Eşit değil mi?	<code>x != 5</code>	<code>false</code>
<code>></code>	Büyük mü?	<code>x > 10</code>	<code>false</code>

Mantıksal Operatörler ve Kısa Değerlendirmesi

fazla mantıksal koşulu birleştirmek için kullanılır:

Operatör	Adı	Açıklama
Operatör	Adı	Açıklama
	Mantıksal VEYA (OR)	Koşullardan en az biri <code>true</code> ise sonuç <code>true</code>
&	Mantıksal VE (AND)	Her iki koşul da <code>true</code> ise sonuç <code>true</code>
!	Mantıksal DEĞİL (NOT)	Koşulun tersini alır (<code>!true == false</code>)

Şik Atama Operatörleri

ik işlem ve atamayı tek bir adımda birleştiren operatörler:

Operatör
Operatör

`+=`

`*=`

`-=`

`/=`

Örnek
Örnek

`x += 5;`

`x *= 2;`

`x -= 3;`

`x /= 4;`

Açılımı
Açılımı

`x = x + 5;`

`x = x * 2;`

`x = x - 3;`

`x = x / 4;`

Artırma ve Azaltma Operatörleri: Önek vs Sonek

```
int a = 5;  
int b = ++a; // Önek (Prefix): Önce a artırılır (6), ardından b'ye atanır (b = 6)  
  
int x = 5;  
int y = x++; // Sonek (Postfix): Önce x'in mevcut değeri atanır (y = 5), sonra x art
```

Önemli Notu: Temel tamsayılarda döngü içinde `++i` ve `i++` aynı optimizasyona uğrar; ancak nesne yönelimli sınıflar (örneğin STL iteratörlerinde) önek `++i` geçici kopya nesne üretmediği için daha verimlidir.

İsim Alanı (Namespace) Mimarisi ve Çakışma Önleme

Kütüphanelerin veya modüllerin aynı isimdeki değişken ya da fonksiyonları tanımlaması durumunda isim çakışmalarını engeller:

```
namespace MotorKutuphanesi {  
    int maxHiz = 100; // Çakışma oluşmaz; farklı isim alanı  
    void baslat() { /* Sensör başlatma */ }  
}  
  
int maxHiz = 255;  
void baslat() { /* Donanım başlatma */ }
```

```
namespace SensorKutuphanesi {
```

Kapsam Çözümleme Operatörü (::) ile Erişim

ni içinde tanımlanmış bir elemana erişmek için :: (Scope Resolution Operator) kullanılır:

```
int main() {  
    // Kapsam operatörü ile doğrudan erişim:  
    return 0;  
    MotorKutuphanesi::baslat();  
    int motorLimit = MotorKutuphanesi::maxHiz;  
  
    SensorKutuphanesi::baslat();  
    int sensLimit = SensorKutuphanesi::maxHiz;  
}
```

`using namespace std;` Direktifinin Riskleri

güç seviyesi kaynaklarda sıklıkla görülen bu kullanım:

```
include <iostream>
using namespace std; // TÜM std isim alanını global kapsama açar!

nt main() {
    cout << "Merhaba\n";
}
```

mladığınız bir değişken veya fonksiyon standart isimlerle çakışır

Çakışması Senaryosu ve Derleyici Belirsizliği

```
include <iostream>
// DEĞERLEME HATASI: std::count fonksiyonu ile global count çakışır!
using namespace std;
    cout << count << endl;
return 0;
/ Projeye ait global değişken:
int count = 10;

int main() {
```

Güvenli İsim Alanı Kullanım Stratejileri

Kapsam Bildirimi (En Güvenli ve Önerilen Standart):

```
std::cout << "Marmara Mekatronik\n";
```

```
cout << "Sistem Hazır" << endl; // Yalnızca cout ve endl açılmıştır
```

Yalnızca İhtiyaç Duyulan Sembolleri Dahil Etme:

```
using std::cout;  
using std::endl;
```

Alanı İlkeleri ve Akademik Standart

İndislik Kuralı:

dosyalarında (`.h` / `.hpp`) KESİNLİKLE `using namespace` kullanılmaz! İlgili başlığı dahil eden tüm kaynak dosyaları `using namespace` alanını kirletir.

Standardı: Bu ders kapsamındaki tüm sunumlarda, laboratuvarlarda ve sınav teslimlerinde `std::cout` , `std::endl` şeklinde açık kapsam bildirme kuralı uygulanır.

Standart Çıktı Akışı: `std::cout` ve `<<`

`cout` karakter tabanlı çıktı akışıdır:

```
include <iostream>
std::cout << "Motor Hızı: " << motorHizi << " RPM, "
nt main() { << "Gerilim: " << gerilim << " V\n";
int motorHizi = 1500;
return 0;
double gerilim = 12.6;

// Zincirleme akış yönlendirme:
```

Standart Girdi Akışı: `std::cin` ve `>>`

den veri okumak için kullanılır:

```
include <iostream>
std::cout << "Ayarlanan Hız: " << hedefHiz << " RPM\n";
return 0;
int hedefHiz;
std::cout << "Hedef motor hızınızı giriniz (RPM): ";
std::cin >> hedefHiz;
```

Standart Girdi Akışında (cin) Hata Yönetimi

ci tamsayı beklenen bir akışa geçersiz karakter (harf/sembol) girdiğinde akış kilitlenir:

```
include <iostream>
if (std::cin.fail()) {
    std::cout << "HATA: Geçersiz tamsayı girdisi algılandı!\n";
} else {
    int sayi;
    std::cout << "Girilen sayıyı giriniz: ";
    std::cin >> sayi;
    return 0;
}
```

di Arabelleği Temizleme (`cin.clear` ve `.ignore`)

girdiden sonra programın yeni veri okuyabilmesi için iki işlem zorunludur:

```
std::cin.ignore(10000, '\n');  
  
if (std::cin.fail()) {  
    std::cout << "Lütfen geçerli bir sayı giriniz.\n";  
    // 1. Akışın hata bayrağını temizle  
    std::cin.clear();  
  
    // 2. Arabellekte biriken hatalı karakterleri satır sonuna kadar temizle
```

Format Manipulator ve Formatlı Çıktı (cout)

Formatlı tabloları ve hassas mühendislik çıktıları için formatlama:

```
// Sabit ondalıklı ve 2 basamak hassasiyet:
#include <iostream>
using namespace std;
int main() {
    double pi = 3.1415926535;
    std::cout << "Pi (2 basamak): " << pi << "\n"; // 3.14

    std::cout << "Pi (4 basamak): " << pi << "\n"; // 3.1416
}
```

Standart Akış Manipülatörleri ve Görevleri

`std::fixed` :

Standartlık gösterimi sabitler; bilimsel (`3.14e+00`) gösterim yerine standart gösterim sağlar.

Bir sonraki çıktının alan genişliğini `n` karakter olarak ayarlar (sağa dayalı tablo basımında kullanılır).
`std::setprecision(n)` :

`std::left` ve `std::right` ek basamak sayısını belirler (otomatik yuvarlama yapar).

Çıktının sütun içerisindeki hizalama yönünü belirler.
`std::setw(n)` :

Hata Akışı: `std::cerr` ile Arabelleksiz Çıktı

Sıfıra ve hata durumlarında standart hata akışı kullanılır:

```
include <iostream>
std::cerr << "[HATA]: Sıfıra bölme yapılamaz!" << std::endl;
return 1; // Hata durum kodu ile sonlandırma
int main() {
    int pay = 10, payda = 0;

    std::cout << "Sonuç: " << pay / payda << "\n";
    if (payda == 0) {
        return 0;
    }
    // Standart hata akışı (Arabelleksiz - Unbuffered)
```

`std::cout` (Buffered) ile `std::cerr` (Unbuffered) Karşılaştırması

`std::cout` (Arabellekli - Buffered):

`std::cerr` (Arabelleksiz - Unbuffered):

`std::cout` verimliliği için kullanılır. Veriler önce tamamlanır ve verimli paketler halinde iletilir; dosyaya yönlendirildiğinde arabellekte bekletilir. `std::cerr` ise hata, uyarı ve arıza telemetrisi için kullanılır. Veriler anında ekrana basılır; arabellekte bekletilmez; program çökse dahi anında ekrana basılır.

Standart çıktıdan ayrı log dosyasına yönlendirilebilir (`./prog 2> hata.log`).

Proje 1: Dört İşlem (Problem Tanımı)

Verilen iki tam sayı alarak toplam, fark, çarpım ve bölüm sonuçlarını hesaplayan programı tasarlayınız. Programın kullanıcı girdisine karşı savunmacı kontrol uygulayınız.

Çarpım: 40

Bölüm: 2.5

İlkinci sayıyı girin: 10

İkinci sayıyı girin: 4

Toplam: 14

Fark: 6

Örnek 1: C++ Çözümü

```
#include <iostream>
```

```
int main() {  
    int sayi1, sayi2;  
    int toplam = sayi1 + sayi2;  
    int fark = sayi1 - sayi2;  
    int carnim = sayi1 * sayi2;  
    std::cout << "Birinci sayıyı girin: ";  
    std::cin >> sayi1;  
    std::cout << "İkinci sayıyı girin: ";  
    std::cin >> sayi2;  
    std::cout << "Toplam: " << toplam << "\n";  
    std::cout << "Fark: " << fark << "\n";  
    std::cout << "Çarpım: " << carnim << "\n".  
}
```

Örnek 2: Daire Alanı ve Çevresi (Problem Tanımı)

Verilen yarıçap değeri olarak dairenin alanını hesaplayınız:

$$\text{Alan} = \pi \cdot r^2$$

```
Yarıçapı girin: 7  
Dairenin alanı: 153.94
```

Örnek Konsol Çıktısı:

Örnek 2: C++ Çözümü

```
include <iostream>
include <iomanip>

int main() {
    const double PI = 3.14159;
    double alan = PI * yaricap * yaricap;
    double yaricap;
    std::cout << std::fixed << std::setprecision(2);
    std::cout << "Dairenin alanı: "; << alan << "\n";
    std::cout << "Yarıçapı girin: ";
    std::cin >> yaricap;
    return 0;
}
```

Örnek 3: Karakter ve ASCII Karşılığı (Problem 1)

Verilen bir karakterden tek bir karakter olarak bu karakterin bellekteki sayısal ASCII kodunu yazdıran programı yazınız.

```
Bir karakter girin: A
```

```
Karakter: A
```

```
ASCII Karşılığı: 65
```

Örnek 3: C++ Çözümü

```
include <iostream>

int main() {
    char karakter;
    std::cout << "Bir karakter girin: ";
    std::cin >> karakter;
    std::cout << "ASCII karşılığı: " << asciiKodu << "\n";

    return 0;
}

// char türünden int türüne güvenli dönüşüm
int asciiKodu = static_cast<int>(karakter);
```

Örnek 4: Sayı İşareti Kontrolü (Problem Tanımı)

Bir tam sayının pozitif, negatif ya da sıfır olduğunu kontrol eden programı tasarlayınız (Hafta 1 örneği için).

Örnek Konsol Çıktısı:

```
Bir sayı girin: -7  
Sayı negatiftir.
```

Örnek 4: C++ Çözümü

```
include <iostream>
```

```
int main() {  
    int sayi;  
    if (sayi < 0) {  
        std::cout << "Sayı negatiftir.\n";  
    } else if (sayi == 0) {  
        std::cin >> sayi;  
        std::cout << "Sayı sıfırdır.\n";  
    } else if (sayi > 0) {  
        std::cout << "Sayı pozitiftir.\n";  
    }  
    return 0;  
}
```

Örnek 5: Harf Notu Belirleme (Problem Tanımı)

Araştırma: Aralığında girilen sınav notunu harf karşılığına dönüştürünüz:

100: A | 80–89: B | 70–79: C | 60–69: D | 0–59: F

Notunuzu girin (0–100): 85

Harf Notu: B

Örnek Konsol Çıktısı:

Örnek 5: C++ Çözümü

```
#include <iostream>
```

```
int main() {  
    int notDegeri;  
    std::cout << "Harf Notu girin (0-100): ";  
    std::cin >> notDegeri;  
    if (notDegeri >= 90) {  
        std::cout << "Harf Notu: A\n";  
    } else if (notDegeri >= 80) {  
        std::cout << "Harf Notu: B\n";  
    } else if (notDegeri >= 70) {  
        std::cout << "Harf Notu: C\n";  
    } else if (notDegeri >= 60) {  
        std::cout << "Harf Notu: D\n";  
    } else if (notDegeri < 0 || notDegeri > 100) {  
        std::cout << "Geçersiz not! 0-100 aralığında bir değer giriniz.\n";  
    }  
}
```

laboratuvar Değerlendirme Ölçütleri

k laboratuvar çalışmaları aşağıdaki akademik ölçütlere göre değerlendirilir:

	Ağırlık	Açıklama
	Ağırlık	Açıklama
nebilirlik	%30	Kod <code>-Wall -Wextra</code> ile derleyici uyarısı vermeden derlenir
kunabilirliği	%20	Anlamlı değişken adları, standart girintileme ve dokümantasyon
Çıktı	%30	İstenen test senaryolarını ve çıktı formatını eksiksiz sağlar
macı Kodlama	%20	Sıfıra bölme, arabellek ve sınır kontrollerinin yapılması

Oratuvar Teslim ve Kod Kalite Standartları

ya Adlandırma Kuralı: `hafta1_<ogrenciNo>.cpp`

msız Çalışabilirlik: Kod tek dosyada derlenebilir olmalı ve `main()` fonksiyonu içermelidir.

Çıktıları: Konsol çalışma çıktıları yorum satırı olarak dosya sonuna eklenmelidir.

demik Dürüstlük: Marmara Üniversitesi Akademik Dürüstlük İlkeleri uyarınca teslim edilen tüm k
nciye ait olması zorunludur.

-'a Başlarken Sık Yapılan Hatalar (1)

Ok Yönlerini Karıştırmak:

Yanlış: `cin << sayi; veya cout >> "Merhaba";`

Doğru: `int toplam = 0; toplam += 5; (değişkene) ve cout << "Merhaba"; (ekrana doğru)`

İnisiyale Edilmemiş Değişken Kullanmak:

Yanlış: `int toplam; toplam += 5;` (Bellekteki çöp değer üzerine ekler!)

-'a Başlarken Sık Yapılan Hatalar (2)

Sayı Bölmesinde Kesir Kaybı:

atalı: `double oran = 1 / 2;` → Sonuç `0.0` çıkar!

ğru: `if (x == 5)` (Eşitlik karşılaştırması yapar)
`----- = 1.0 / 2;` veya `static_cast<double>(pay) / payda`

atalı Virgül ve Eşitlik Karışıklığı:

atalı: `if (x = 5)` (Atama yapar ve koşul daima `true` kabul edilir!)

Program ve Mimari İlkeleri: OOP ve Derleme

li Değerlendirme İlkeleri (1):

OOP Kavramları: Sınıf (şablon) ile Nesne (somut varlık) ayrımı, Kapsülleme (veri gizleme), Kalıtım ve Polimorfizm temel sınav ve mülakat konularıdır.

Derleme Aşamaları: Hangi hatanın hangi aşamada (Önişlemci, Derleyici, Bağlayıcı) üretildiği iyi kavranmalıdır: Başlık eksikliği önişlemcide; sözdizim ve tip hatası derleyicide; tanımsız gövde (undefined reference) bağlayıcıda saptanır.

Yazma ve Bellek İlkeleri: Bellek ve Akış

li Değerlendirme İlkeleri (2):

Tür Dönüşümleri: `int / int` tamsayı bölme tuzağı ve `static_cast` kullanımı klasik bir mühendislik sınav sorusudur.

Değişken İlkendirme: Yerel değişkenlerin çöp değerleri (garbage value) ve C++11 `{}` daralma (narrowing) konuları bilinmelidir.

`cin` Akışı: Boşluk karakterlerinin davranışı ve `cin.fail()` akış kilitlenmesi hata kontrolünde temel değerlerdir.

laftanın Özeti

Paradigması: Gerçek hayat sistemleri nesnelerle modellenir. Sınıf projesidir, nesne fiziksel varlıklar. Özellikler: Kapsülleme, Kalıtım, Çok Biçimlilik, Soyutlama.

Değişkenler & Tipler: Bellekte her değişkenin boyutu ve adresi vardır. Değişkenler daima ilkendirilmelidir. Dili: C'nin doğrudan donanım hizını (Zero-overhead), nesne yönetimli mimariyle birleştirir. (;).

Yapılandırma Hattı: Önışlemci → Derleyici → Amaç Kod (.o) → Bağlayıcı → Binary.

İşlevler: `std::cout` ekrana yazar (<<), `std::cin` klavyeden okur (>>), `std::cerr` hata akışı.

Standartlar: Tip dönüştürmelerinde `static_cast`, formatlamada `<iomanip>` ve diğer.

Kaynaklar ve İleri Okuma

k

İlgili Bölüm

& Deitel, *C++ How to Program*

k

Bölüm 1–2 (Giriş, Veri Tipleri, G/Ç)

alick, *Data Structures Using C++*

İlgili Bölüm

erence.com

Bölüm 1–2 (Temeller)

oustrup, *The C++ Programming Language*

`iostream` , `iomanip` , temel tipler
Bölüm 1–3

olarına KAYNAKCA/ klasöründen ulaşabilirsiniz.

6. Hafta Konuları

Denetimli Kontrol Yapıları, Karar Algoritmaları ve Döngüler

Denetim mekanizmaları: `if`, `else if`, `else` mantığı

Seçim: `switch-case` yapısı ve `break` kullanımının önemi

İşlemler: `for`, aralık tabanlı `for (: )`, `while`, `do-while` döngüleri

Kontrol: `break` ve `continue` ifadeleri

Pratik Uygulamaları: DC motor kademe seçimi, eşik denetimi ve Sonlu Durum Makinesi (FSM)